Student Marks Matrix Analysis

Python Program

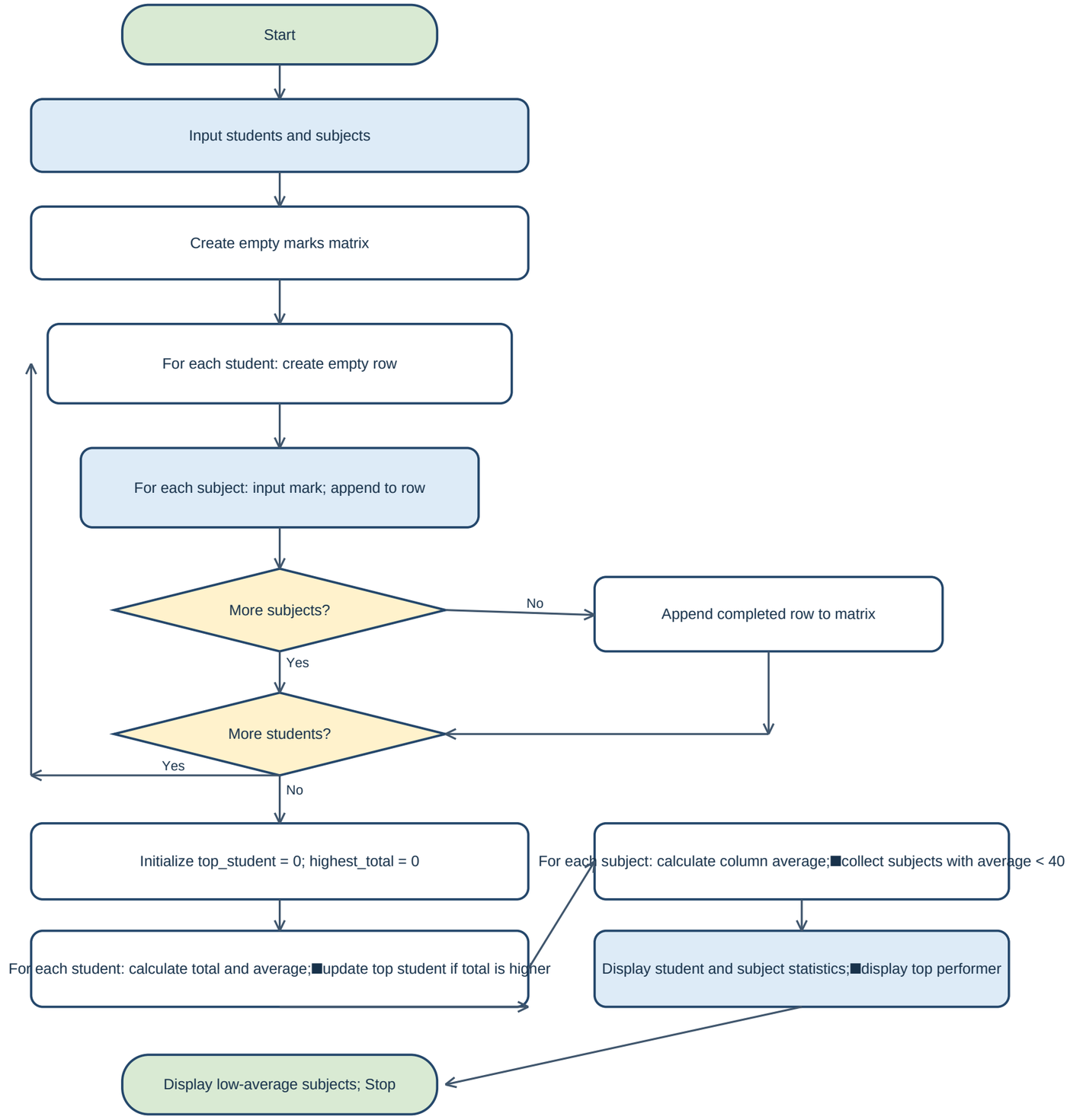
Assigned syllabus matter: Python solutions using only the assigned syllabus: computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations, and unpacking.

```
# Student Marks Matrix Analysis

students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))

marks = []

print("\\nEnter marks:")
for i in range(students):
    row = []
    print(f"\\nStudent {i + 1}")

    for j in range(subjects):
        mark = int(input(f"Subject {j + 1}: "))
        row.append(mark)

    marks.append(row)

# Row-wise Statistics (Student Performance)
print("\\nStudent Statistics")

top_student = 0
highest_total = 0

for i in range(students):
    total = sum(marks[i])
    average = total / subjects

    print(f"Student {i + 1}: Total = {total}, Average = {average:.2f}")

    if total > highest_total:
        highest_total = total
        top_student = i + 1

# Column-wise Statistics (Subject Performance)
print("\\nSubject Statistics")

low_average_subjects = []

for j in range(subjects):
    subject_total = 0

    for i in range(students):
        subject_total += marks[i][j]

    subject_average = subject_total / students

    print(f"Subject {j + 1}: Average = {subject_average:.2f}")

    if subject_average < 40:
        low_average_subjects.append(j + 1)

# Top Performer
print(f"\\nTop Performer: Student {top_student}")
print(f"Highest Total Marks: {highest_total}")

# Low Performing Subjects
if len(low_average_subjects) > 0:
    print("\\nSubjects with Low Average Performance:")

    for subject in low_average_subjects:
        print(f"Subject {subject}")
else:
    print("\\nNo subjects have low average performance.")
```

Student Marks Matrix Analysis

Program Flowchart

STUDENT MARKS MATRIX ANALYSIS — FLOWCHART



The matrix is built using nested loops, then analyzed row-wise for students and column-wise for subjects.

Student Marks Matrix Analysis

Project Matter, Algorithm and Pseudocode

Assigned Syllabus Matter

Python solutions using only the assigned syllabus: computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations, and unpacking.

1. Aim

To analyze students' marks stored in a two-dimensional list, calculate each student's total and average, calculate each subject's average, identify the top-performing student, and list subjects whose average mark is below 40.

2. Description

The program accepts the number of students and subjects, then reads each student's marks and stores them as rows in a nested list (matrix). It performs row-wise processing for student totals and averages, followed by column-wise processing for subject averages. It also identifies the highest student total and records subjects with averages below 40.

3. Input

Input	Purpose
Number of students	Number of student records.
Number of subjects	Number of subjects per student.
Marks	Integer mark for each student in each subject.

4. Data Structure Used

The nested list marks stores one row per student. Each row contains that student's marks for all subjects. Row-wise access is used for student statistics; marks[i][j] accesses the mark of student i in subject j for subject statistics.

5. Algorithm

1. Read the number of students and subjects.

2. Create an empty marks matrix.

3. For each student, create a row; input each subject mark and append it to the row.

4. Append each completed row to marks.

5. Initialize top_student and highest_total to 0.

6. For each student, calculate and display the total and average; update the top student if the total is greater.

7. For each subject, add its marks across all students and calculate the subject average.

8. If a subject average is below 40, add its subject number to low_average_subjects.

9. Display the top student and highest total.

10. Display low-average subjects, or display the message that none exist.

11. Stop.

6. Pseudocode

```
START
  INPUT number_of_students, number_of_subjects
  CREATE empty 2D list marks

  FOR each student i
    CREATE empty row
    FOR each subject j
      INPUT mark
      APPEND mark to row
    END FOR
    APPEND row to marks
  END FOR

  SET top_student = 0
  SET highest_total = 0
  FOR each student i
```

```
    CALCULATE total of marks[i]
    CALCULATE average = total / number_of_subjects
    DISPLAY student number, total, average
    IF total > highest_total THEN
      SET highest_total = total
      SET top_student = i + 1
    END IF
  END FOR

  CREATE empty list low_average_subjects
  FOR each subject j
    SET subject_total = 0
    FOR each student i
      ADD marks[i][j] to subject_total
    END FOR
    CALCULATE subject_average = subject_total / number_of_students
    DISPLAY subject number and subject_average
    IF subject_average < 40 THEN
      APPEND j + 1 to low_average_subjects
    END IF
  END FOR

  DISPLAY top_student and highest_total
  IF low_average_subjects is not empty THEN
    DISPLAY each subject in low_average_subjects
  ELSE
    DISPLAY "No subjects have low average performance"
  END IF
STOP
```

7. Output

The program displays every student's total and average, every subject's average, the top performer and highest total, and either the subjects with averages below 40 or a message that no subject has a low average.

8. Important Notes

• A subject is listed only when its average is strictly less than 40; an average of exactly 40 is not included.
• The program assumes students and subjects are greater than zero to avoid division by zero.
• Ties for the highest total retain the first student encountered. Because highest_total starts at 0, if every student's total is 0, top_student remains 0 in the supplied code.

9. Conclusion

This program demonstrates nested lists, loops, arithmetic operators, conditional statements, and list operations to summarize student and subject performance and identify areas needing attention.